

Application: Shift_quality_happens

Students

- Aleksandr Uktamovich Sorokin - also1002@stud.ek.dk
- Lucas Levy l'Espagnol Chantelou - luch0002@stud.ek.dk
- Jafar Alsukafi - jaal0002@stud.ek.dk
- Sebastian Holger Drumm - sedr0001@stud.ek.dk
- Viggo Møhring Beck - vibe0002@stud.ek.dk

Brief description of the application

Shift_quality_happens is a shift planning application used to manage employees, shifts, assignments and shift swaps. The application supports planning work schedules while checking important business rules, such as avoiding overlapping shifts, validating rest periods between shifts, handling night and ensuring that shift swaps and assignments follow the intended rules. The testing activities will focus on whether the system behaves correctly from both a technical and user-oriented perspective.

Expected testing activities

- **Black-box test design:** Use equivalence partitioning, boundary value analysis and decision tables to derive test cases from business rules, including positive and negative cases.
- **Static testing:** Review selected requirements and test basis, and use static code analysis/linting to identify syntax, style, maintainability and possible code quality issues before execution.
- **Unit testing:** Test central business logic in isolation, especially rules for shift overlap, minimum rest periods, night shift calculation, weekly work limits and validation of date/time input.
- **White-box testing:** Use statement/branch coverage to check whether important decisions in the code are exercised, especially validation paths and rule-based conditions.
- **Integration testing:** Test interaction between backend services, API endpoints and the database, including creation, update and cleanup of employees, shifts, assignments and shift swap data.
- **API testing:** Verify relevant endpoints for correct HTTP status codes, validation errors, authentication/authorization behaviour and JSON response formats.
- **Stress tests:** The process of pushing an application beyond normal operating limits to evaluate its stability, performance, and failure handling under extreme load. Smoke, load, stress, spike, soak tests have been made.
- **End-to-end/UI testing:** Automate representative user flows where possible, such as login, viewing schedules, creating shifts, assigning employees.
- **Database integration testing:** Use a test database or controlled test data setup, ensuring that test data is isolated and cleaned up when necessary.
- **Regression testing and reporting:** Re-run automated tests after changes, collect code coverage/test reports and document discovered defects, limitations and test results.
- **Acceptance-oriented testing:** Compare selected system behaviour against the expected user needs for a shift planning system, including whether rules are understandable and useful for the intended users.